Unit 3

Inheritance

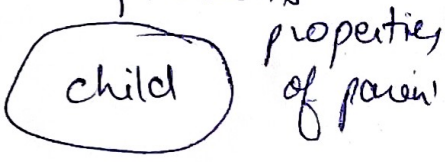
Using already built things.
Reusability feature is inheritance.
If someone asks you to make a software then will you start coding for the whole software from scratch or you will look if there is some built software of same type and will incorporate its features in your new one.

Inheritance is one of the most important feature of OOP. This provides us the capability to reuse the preexisting code to our new project/s.
Its always better to reuse the pre existing codes rather then developing all over again. This reusability feature is incorporated by the concept of Inheritance.

Real life Entity

Types of Inheritance

- Single Inheritance
- multiple Inheritance
- multilevel Inheritance
- Hybrid Inheritance
- Hierarchial Inheritance.

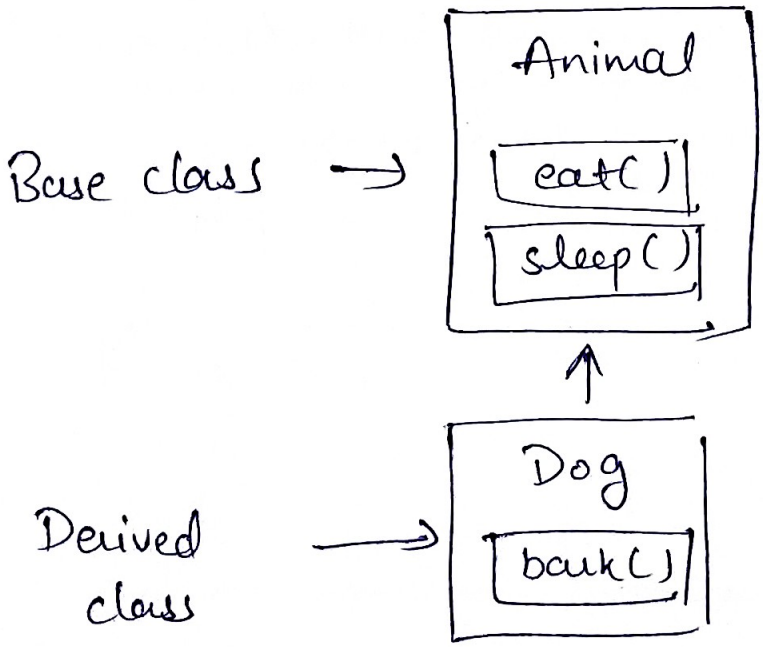


Capability of a class to derive properties and characteristics from another class is called Inheritance.

It is a feature or a process in which new classes are created from the existing class.

The new class created is called derived class or child class and the existing class is called base class or parent class. The derived class now is said to be inherited from the base class.

To inherit from a class, use symbol :



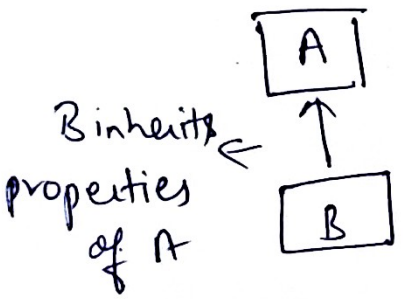

```
//base class
class Animal {
public:
Animal()
{
cout << "This is an animal";
}};
class dog : public Animal
{

};
```

```
int main()
{
dog d1;
return 0
}
```

Creating object of sub class will invoke constructor of base class. ←

Single inheritance

When one base class is being derived by a single sub class, then this is called single Inheritance. It means, when there is one base class and it is being derived by another single derived class, then it is called single inheritance.



Base class / parent class / Super class

Derived class / child class / sub class

```
#include <iostream.h>
#include <conio.h>
class A
 {
 protected:
  int x;
 public:
  void input()
   {
    cout<<"\n enter no:";
    cin >>x;
   }
};
```

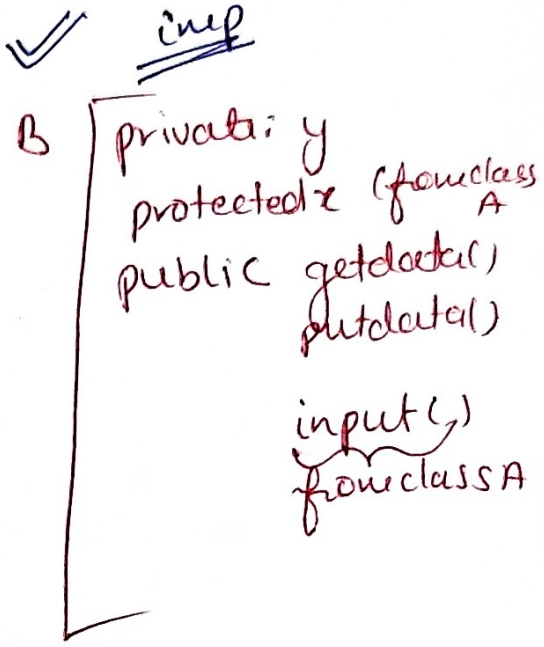
A | Protected: x
public:
void input()

```
class A
 {
  Private:
int x;
public:
 void input()
  {
   cout<<"enter no";
   cin >>x
  }
int getx()
 {return x;
 };
```

It count private instead of public

```
then in void putdata
void putdata()
{
cout<<"addition"<<
      getx()+y;
```

```
Class B : public A
{
  int y;
  public:
   void getdata()
   {
   cout << "enter no";
    cin >> y
   }
   void putdata()
   {
    cout << "\n addition = " <x+y;
    }

   };
int main()
  {
   B aa;
   aa.input();
   aa.getdata();
   aa.putdata();
   getch();
   return 0;
   }
```

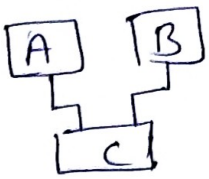


★ Object in main is always made of derived class.

Output

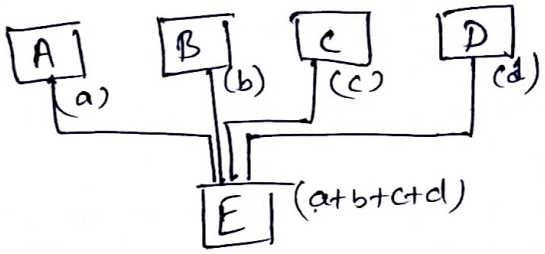
```
enter no
   1
enter no
   2
addition
   3
```

Multiple Inheritance



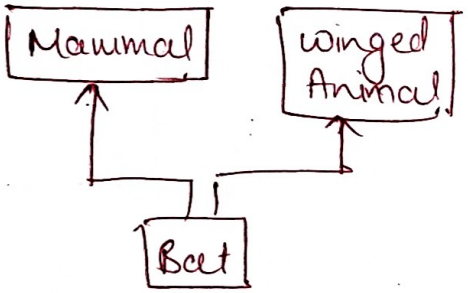
When there are more than one base class are being inherited by single derived class, then it is called concept of multiple inheritance.

for example



A class can also be derived from more than one base class using a comma separated list.

```
class mychildclass : public myclass, public myclass1
  {
    };
```




```
# include <iostream.h>
# include <conio.h>
 class A
   {protected:
     int a;
```

```
public:
  void input ()
   {
    cout << "\n enter no.";
    cin >> a;
   }
};
class B
 {
 protected:
    int b;
 public:
  void getdata()
  {
   cout << "\n enter no";
   cin >> b;
  }
};
```

A [protected a
public input

// Both classes A & B are independent

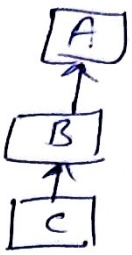
B [int b protected
public getdata

```
class C : public A , public B
  {
  public:
   void addition()
   {
    cout << "\n addition =" << a+b;
   }
};
```

C [Protected : a & b
public : input
getdata
addition

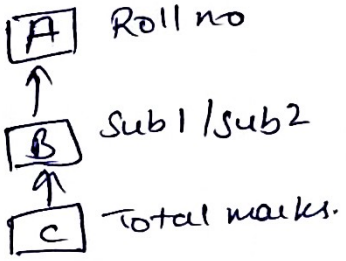
```
int main ()
{
  C aa;
aa.input ();
aa.getdata();
aa.addition();
getch();
return 0;
}
```

multilevel inheritance



(level by level inheritance)

Multilevel inheritance is a type of inheritance in which one super/base class is derived by a child class and that this child class is further derived by another child class and so on. This type of inheritance is called multilevel inheritance.




```
# include<iostream.h>
# include <conio.h>
class A
{protected:
 int roll;
 Public
   void getroll()
    {cout <<"enter Roll";
     Cin >>roll;
 }
  void putroll()
  {cout <<"\Roll no:="<<roll;
   }
};
class B: public A
 {Protected:
  int sub1, Sub2;
  Public
    void getmarks()
    {cout<<
     Cin >>Sub1 >>Sub2;
    }
    Void putmarks()
  {cout <<"In mark1="<<Sub1 << "In mark2="<<Sub2;
  } };
```

A
- Protected roll
- Public getroll()
- putroll()

B
- protected sub1, sub2
- Public getmarks()
- putmarks()

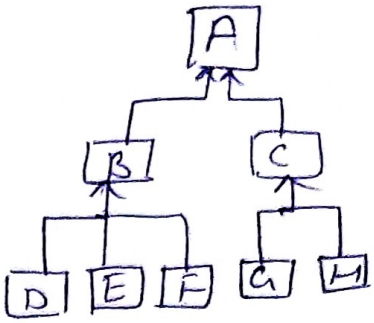
```
class C : Public B
{
  int sptm;
  public:
   void getsptm()
    {cout << " enter sports marks ";
     cin >> sptm;
    }
   void total()
   {
    cout << "total marks = ".
    putroll().
    putmarks();
   cout << "sports mark = " << sptm;
   cout << "\Total marks: " << sub1+sub2+sptm;

   }
};
int main()
 {
  C aa;
  aa.getroll();
  aa.getmarks();
  aa.totale();
  aa.getsptm();
  aa.total();
  getch();
  return 0;
  }
```

when we call a function inside a function then there is no need of object.

aa

- private: sptm
- protected: sub1, sub2, roll
- public: getsptm(), total(), getmarks(), putmarks(), putroll(), getroll()

Hierarchial Inheritance



When one base class is being inherited by multiple derived classes, then this is called the concept of Hierarchial Inheritance

Program

```
# include <iostream.h>
# include <conio.h>
Class A
 {
  Public :
    void message( )
    {
     cout<<"\n welcome to inheritance";
    }
 };
class B : Public A
  {
   public:
   void display( )
    {
     cout<<"\n Inside class B";
    }
 };
class C : public A
  {
   public:
    void putdata( )
   {cout<<"\n inside class C";
   }
};
```

A [Public
void message

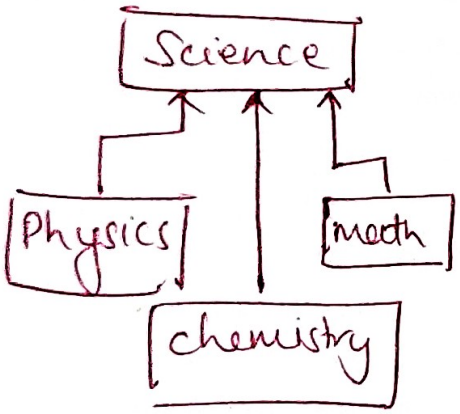
B [Public
void message
void display

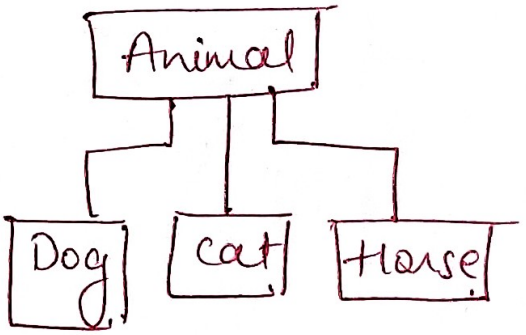
C [Public
putdata
message

```
int main ()
{
 B aa ;
 C bb;
 aa.display ( );
 aa. message ( );
 bb. putdata ( );
 bb. message ( );
 getch ( );
 return 0;
}
```

Output

Inside class B
welcome to Inheritance
Inside class C
welcome to Inheritance

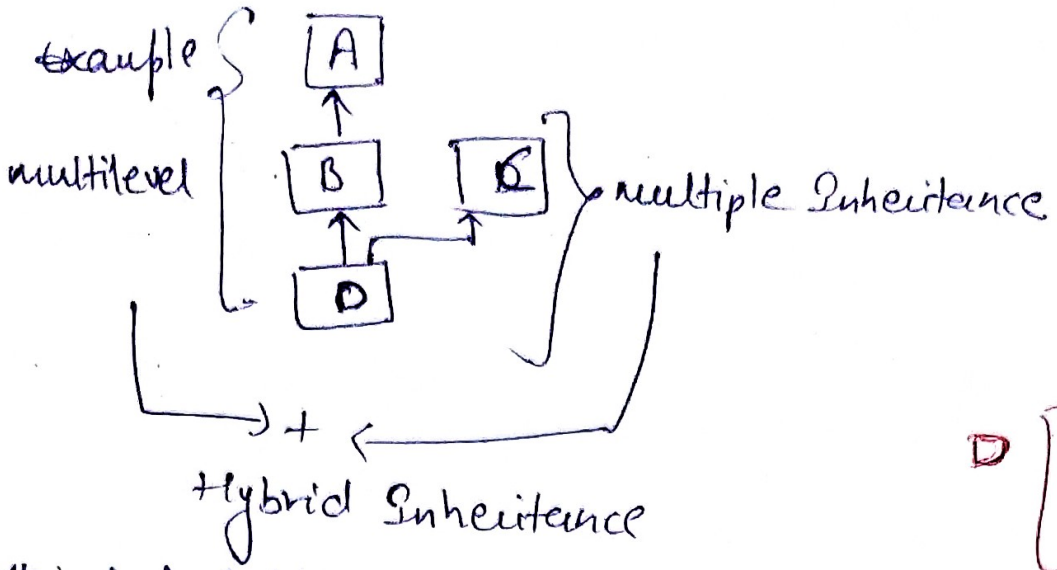




Hybrid Inheritance

14 basic types
① single ② Multiple
③ Multilevel ④ Hierarchical

When we combine the concepts of two or more basic inheritance types, then the new type of Inheritance is referred to as Hybrid Inheritance



D [print
display, putdata
message

```
#include <stdio.h>
class A
 { public:
   void putdata()
    {
    cout<<"inside class A";
    }
 };
class B: public A
   {
    public
     void display()
     {cout<<"inside class B";
      }
   };
class C {
    { public
      void message()
      {cout << "inside class C";
       }
   };
 class D: public B, public C
   { public:
      void print()
       { cout <<"In Inside class D";
       } };
```

```
int main()
{
 D dd;
  dd.print();
  dd.display();
  dd.putdata();
  dd.message();
 return 0;
}
```

Output

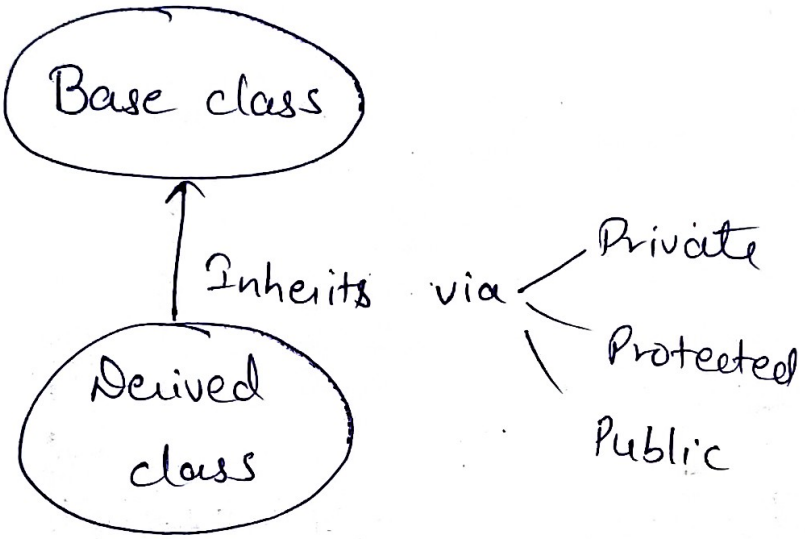
Inside class D
inside class B
inside class A
inside class C

Defining Derived class

Access specifies

Private
Protected
Public

} In Inheritance are called as visibility modifiers

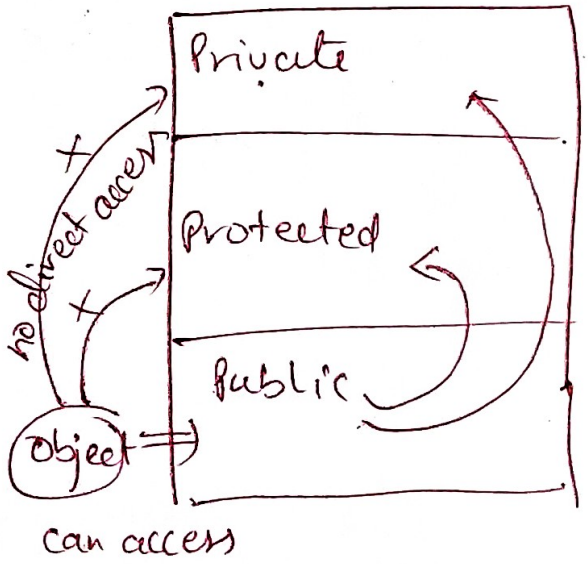


Base class can be inherited by private, protected or public methods.

By default inheritance is private.

class & object

Base / Derived	Private	Protected	Public
Private	X	private	Private
Protected	X	protected	protected
Public	X	protected	public



class abc

Private a, b
Protected c, d
Public getdata putdata

class xyz

private m, n, c, d getdata() putdata() Protected, o Public - input() output()

class xyz : private abc

{ —
—
—
}

inheriting ~~Private~~

xyz

private m, n c, d getdata putdata ⇐ Protected o Public input(), output()

Private will not get inherited but protected & public of abc will be in private of xyz

class xyz : protected abc

{ —
}

xy

xyz

private m, n protected, o c, d getdata, putdata, public input output

class xyz : public abc

xyz

privdta m, n protected o c, d Public input() output getdata() putdata()

Method Overriding in C++ // Function Overriding

Suppose we have two classes having same name function and also having same signature, so in this case, obviously object of derived class is created and when calling that function, the func" of derived class is called and executed. So here the function of derived class has overriden the function of base class. This concept is called method overridiy or function overridiy.

```
#include<iostream.h>
class A
{
 public:
  void display()
  {
   cout<<"In Base class";
  }
};
class B: public A
 {public:
   void display()
  {
  cout<<"In Derived class";
A::display();
 };
int main()
{
 B aa;
 aa.display()
 return 0;
}
```

Output
Derived
Base

Here when Baa will call display() then it will first of all call within its own class, ∴ display() in class A will be overridden by display() function of class B
This is called Overridiy.

A::display() ; was first solution
But I dont want to do this.
So another method to resolve overridy is we will do some changes in main functio

```
int main()
{
 Baa;
 aa.display(); aa.A::display();
 return 0;
}
```

Constructor Handling in Base class

If there are constructor is Base class then how we handle these constructors.
why? Because constructors are called when we make object of the class, but in inheritance object is always made up of Derived class not of Base class. So if there is constructor in base class then how we will handle it.

```
#include <iostream.h>
class A
{protected:
 int a;
 public:
     A(int x)    // constructor
     {
      a=x;
     }
     void display()
    {
    cout<<"\n A"<<a;
    }
};
class B
{protected:
  int b;
  public:
     B(int y)
     {b=y;
     }
     void putdata()
    {cout<<"\n B="<<b;
    }
class C: public A, public B
{int c;
  public: C(int p, int q, int r):
                 One for c, one for B &
                    one for A
                  A(p), B(q)
     {
       c=r;
     }
   void show()
   {cout<<"\n c="<c;
    }};
```

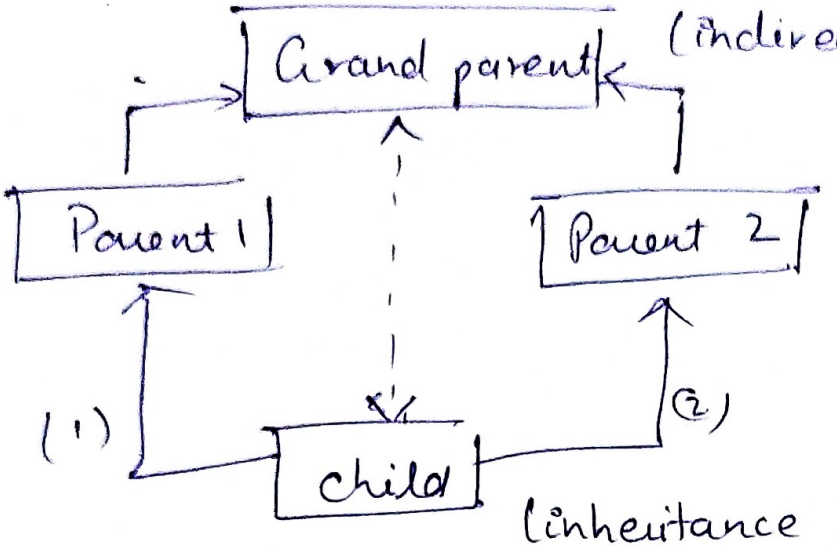
```
class A
 {
   =
   A() //constructor
   {=
   }
};
class B: public A
 {=
  =
 }
int main()
 {
   B aa;
    =
 }
```

constructor is called when object is created, but if object of base class is not created then how we will handle constructor of base class.

Derived class should always have a constructor if base class have constructors because constructors of Derived class only passes value for constructors for base class.
So it is must to put a constructor in derived if they are present in base

```
int main()
{
  C aa (10,20,30);
  aa.display();
  aa.putdata();
  aa.show();
  return 0;
}
```

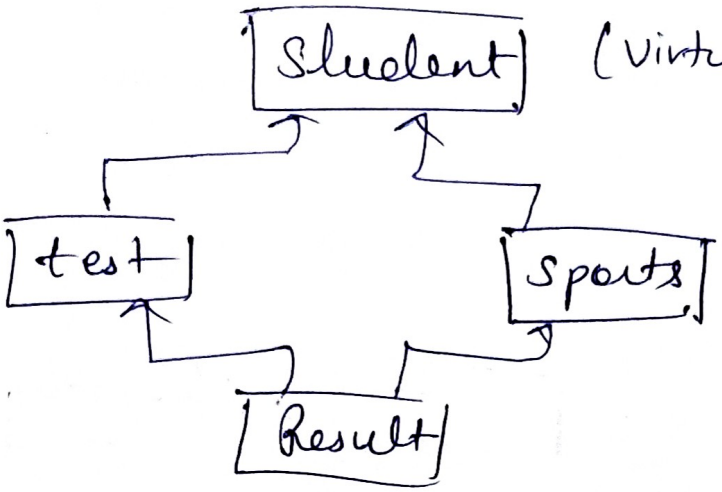
Virtual Base class



child will have duplicate set of members inherited from the grandparents.
So duplicacy of property is happening. (Grandparents Properties)
duplicacy of inherited member due to multiple path can be avoided by making a common base class or virtual base cass

Declaration

```
class A
{
  ---
};
class B1 : virtual public A
{
  ---
};
class B2 : public virtual A
{
  ---
};
class C : public B1, public B2
{
  ---
};
// only one copy of A will be inherited.
```



when a class is made as virtual Base class C++ takes necessary care to see that only one copy of the class is inherited, regardless of how many inheritance path is followed.

~~Abstract class~~

Virtual function

~~Virtual Base class~~ (Late Binding / Early Binding)

Base class shows that Topic is related with Inheritance.

1. Virtual is a keyword in C++

Virtual
not physically existing as such but made by s/w to appear to do so.

```
#include <iostream.h>
class A
{public:
  void show()
   {
    cout << "\n Base class";
   }
};
class B : public A
 { public:
    void show()
     {cout<< "\n Derived class";
     }
 };
```

making ptr of Base class & obj

```
int main()
{
 A *bptr; //base class ptr
 A aa;
 bptr = &aa;
 bptr -> show();
 return 0;
}
```

output will be of Base class

```
int main()
 {
  B aa;
  aa.show();
  return 0;
 }
```

doing method overriding

aa.A::show() called early binding

for calling show() of base class

Pointer is always made of base class.

~~So~~ A *bptr

now we to make an object so suppose we make aa of A A aa;

now we associate ~~it~~ pointer with object

bptr = &aa;

now we call show() using bptr

bptr -> &show();

Now why virtual came

Since object is of base class

pointer is of base class

then that base pointer is associated with that object

and now we are calling our show function with that base pointer then base class will be called.

Now twist comes when pointer is of base class but object is made up of derived class

```
int main()
{
  A *bptr;
  B aa;
  bptr = &aa;
  bptr -> show();
  return 0;
}
```

//still output will be of base class because ptr is of base class

So this is problematic for this we use keyword Virtual.

Concept of Virtual func^n is also called Runtime polymorphism

now we add

```
class A { public:
    virtual void show( )
        {
          —
        }
};
```

So the o/p will be of derived class because virtual tells us that you will be creating pointer of base class but ~~that~~ through that pointer, the object of whatever class you are referring to, ~~will be called~~. function of that class will be called.

Virtual function fascilitates that Based on the object, the class which contains the object, function of that class will be called.

This happens at Runtime of pln. so this is called late binding (because pln decides on runtime that which func^n will be called)

* A Virtual func^n is redefined in derived class.
* when a Virtual func^n is defined in base class, then the pointer to base class is created, on the basis of type of object assigned, the respective class func^n is called.

Rules

- Virtual func^{n} must be member of same class.
- They cannot be static membe.
- They are accessed by using object pointer A *bptr
- VF can be friend of other funcn
- VF in a base class must be defined, even though it may not be used.
- We cannot have virtual constructors but we can have virtual destructors.

UNIT IV

Static Data Member

Static data member & static member functions

What are static data members?

Static data members are also variables but with some properties.

1) static data member is initialised to 0, whenever the first object of its class is created. No other initialisation is permitted

```
Static int x ;  ← Syntax
```

2) for making any data member static, we use static keyword.

3) Only one copy of static data member is created and shared by all.

x will not be part of aa, bb, cc but value of x can be used by all aa, bb, cc.

Data member

```
class demo
{static int x
 int a,b;
public:
 int c;
 void getdata();
 void putdata();
};
```

demo aa, bb, cc;

independent

4) Its visibility is entire program. whether it is made in class but it can be used in whole program.

```
# include <iostream.h>
# include <conio.h>
class demo
 {
  int x,y;
  static int z;
  public:
     void getdata (int a, int b)
     {
       x= a;
       y = b;
       z= z+1;
     }
    void putdata ( )
     {
       cout << " x=" <<x << "\t y =" <<y "\tz" =<<z;
     }
 };
int demo :: z;   // compulsory line sometimes asked in o/p based Question
void main ( )
 {
  demo aa , bb ;
  aa. getdata (5, 10);
  bb. getdata (20, 30);
  aa. putdata ( ) ;
  bb. putdata ( );
getch ( ); }
```

z 0 1 2

aa: x 5, y 10, getdata, putdata

bb: x 20, y 30, getdata, putdata

Static Member function

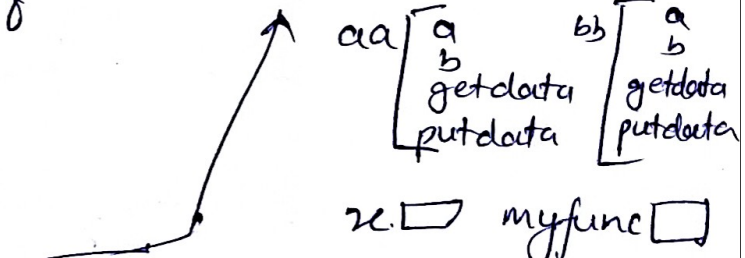
① Static member func^{n} can access only static data members.

② as static data member is not a part of any object same way static member function is not a part of any object. It is independent.

③ Since it is not the part of any object ~~as~~ it is called using the class name :

ex `xyz :: myfunc()`

```
class xyz
{
  int a, b;          // data member
  static int z;      // static data member
public:
  void getdata();    // member func
  void putdata();
  static void myfunc();   // static member function
};
xyz aa, bb;
```




```
# include <iostream.h>
# include <conio.h>
class demo
  {
    int x;
    static int y;
    public :
    void getdata (int a)
```

```
{
 x=a;
 y=y+1;
}
void putdata()
{
 cout<<"\n x="<<x<<"\t y="<<y;
}
```

```
static void abc()
{
 cout<<x;
}
```

error

cannot use normal variables in static func^n

```
{
 cout<<"\n y = "<<y;
};
int demo::y; // compulsory
int main()
{
 demo aa
 aa.getdata(10);
 aa.putdata();
 aa.abc();
demo::abc();
getch();
return 0;
}
```

X //error because it is a static member func^n so can't call it with an object

aa: x, getdata(), putdata()

y: 01 static

~~Static Member function~~

Constant data Members

Const keyword is used to define the constant value that cannot change during program execution.

means if we once declare a variable as the constant in program, the variable's value will be fixed and never be changed.

If we try to change the value of const type variable, it shows an error message in the program.

Syntax

```
const int a;
```

```
# include <iostream>
# include <conio.h>
int main()
{
const int num=25; // declare value of constant
num = num+10;
return 0;
}
```

num =

```
# include <iostream.h>
# include <conio.h>
 int main()
 {
  const int x = 20;
  const int y = 25;  // declare variables
  int z;

  z = x+y;
  cout << " sum of x & y is " << z << endl;
  return 0;
  }
```

Note while declaration of const variable in the c++ programming, we need to assign value of defined variables at the same time; else it shows compile time error.

Constant Data members of class

Data members are like variables that is declared inside a class, but once the data member is initialized, it never changes, not even in constructor or destructor. Constant data member is initialized using const keyword before data type inside the class. The const

data members cannot be assigned the values during its declaration; however they can assign the constructor values.

```
# include <iostream>

class ABC
{
 public:
 const int A;   //use const keyword to declare const data member
 ABC(int y): A(y)   //create class constructor
 {
 cout << "The value of y: " <<y << endl;
 }
};
 int main()
{
 ABC obj(10); //here obj is object of class ABC
 cout << "The value of constant data member
          'A' is: " << obj.A << endl;
 return 0;
}
```

O/p

value of y: 10

The Value of constant data member 'A' is: 10

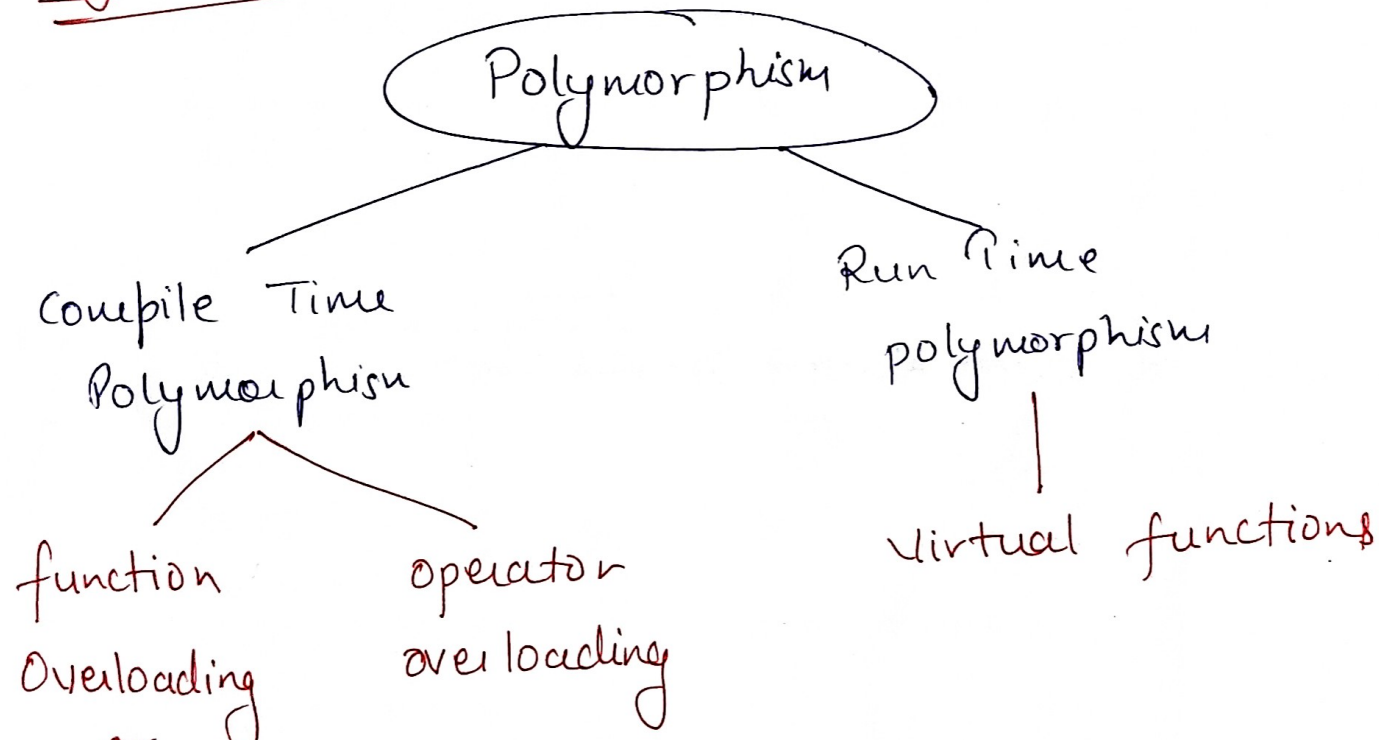
Polymorphism

most important

(feature of OOP)

It is one of the most important feature of OOP which simply implies one name multiple form.

Poly (more than one)



Operator Overloading

+operator a+b → adding 2 int
└ adding 2 floats

arithmatic, relational, increment, decrement

built in work of + is to add 2 nos

but we can add 2 objects with +, so here + is taking another form.

Operator overloading is one of most important feature of C++ which is a type of polymorphism.
Using the concept of operator overloading we can overload any built in operators, i.e we can assign a new definition to an existing operator.

ex + is used to add built in data types

a+b=c
5+4=9

but we use aa+bb=cc
here aa, bb and cc are ~~built~~ in objects
So if we write this normally, it will show an error, as + is used for built in data type not for user defined data type.
But if we overload that + sign, i.e if we assign different work to + operator then it can be used to add two objects.
we can do this with any operator.

Some operators can not be overloaded

1. Scope resolution operator (::)
2. class member access operator (.*)
3. size of operator (sizeof)
   ↓ tells size of datatype in bytes
4. Conditional operator (?:)

Overload (+) operator (-) (*) (/)

```
#include<iostream.h>
class demo
{
 int a;
 public:
    void getdata()
    {
     cout<<"\n enter a no:";
      cin>>a;
    }
    void putdata()
    {
    cout<<"\n value ="<<a;
    }
 demo operator+(demo bb)
  {
    demo cc;
    cc.a = a+bb.a;
    return cc;
 };
```

if you want below to be correct
cc = aa+bb
means you are calling + operator as a function with the reference of aa and passing bb as a parameter. and return of this function is stored in cc

(waiter)
with argument with return.

```
int main( )
{
demo aa, bb, cc;
   aa.getdata( );
   bb.getdata( );
   cc = aa + bb;
   aa.putdata( );
   bb.putdata( );
   cc.putdata( );
   return 0;
}
```

aa[a
bb[a
cc[a

5
10

```
demo  operator + (demo bb)
 ↓          ↓        ↓
returntype  function  argument
            name
{
   demo cc;
   cc.a  =  a  +  bb.a;
    ↓       ↓      ↓
  a of cc  a of aa  a of bb
```

4/11/22

Overloading ++ / -- Operator

increment decrement

a++ ✓
a-- ✓

aa++ ✗
↓
object

```
#include <iostream.h>
class demo
{
  int x;
  public:
   void getdata( )
   {cout <<"\n value of Enter no";
    cin >> x;
   }
   void putdata( )
   {
    cout <<"\n value of x =" <<x;
   }
→ Return type void bcoz NANR function
void operator++( )
   {
    x = x+1;
   }
};
int main( )
{
 demo aa;
 aa.getdata( );
 aa.putdata( );   // original value of x
 ++aa;            // value after increment
 aa.putdata( );
 return 0; }
```

NRNA
a a++
++ aa
aa[x = 5

++ is called with reference of aa
++ aa
Prefix operator
NO argument
NO Return

NANR

for postfix aa++

```
void operator++(int)
{
  x = x+1;
};
int main()
{
  demo aa;
  aa.getdata();
  aa.putdata();
  aa++;
```

Syntax

++aa

// a=a+1;
↑ needs integer

Overloading Assignment Operator (=)

+=, -=, *=, /=

```
#include<iostream.h>
class demo
{
  int a;
  public:
    void getdata()
    {
      cout<<"\n Enter a no";
      cin>>a;
    }
    void putdata()
    {
      cout<<"\n value="<<a;
    }
```

```
Void operator=(demo bb)
{
  a = bb.a;
};
int main()
{
demo aa,bb;
bb.getdata();
 aa=bb;
aa.putdata();
bb.putdata();
return 0;
}
```

// here a is a of aa as we are calling = with aa

aa=bb; ✓
To be true calling = with reference of aa and passing bb as an argument but no Return

WANR

aa[a bb[a=10

aa = bb;

a = b
b = b

```
int a,b
public
 voidgetdata()
{
cout<<"\n Enter 2 no";
cin>>a>>b;
}
Void putdata()
{
 cout<<"\na="<<a
     <<"\nb="<<b;
}
```

```
void operator=(demobb)
{
  a=bb.a;
  b=bb.a;
}};
int main()
{
demo aa,bb;
bb.getdata();
aa=bb;
aa.putdata();
bb.putdata();
```

bb=aa

aa[a b]

bb[a b]

```
void operator=(demo bb)
{
  a = bb.a;
};
int main()
{
demo aa,bb;
bb.getdata();
 aa=bb;
aa.putdata();
bb.putdata();
return 0;
}
```

// here a is a of aa as we are calling = with aa

aa=bb; ✓
To be true calling = with reference of aa and passing bb as an argument but no Return

WARNR

aa[a bb[a=10

aa = bb;

a = b

b = b

```
int a,b
public
 void getdata()
{
cout<<"\n enter 2 no";
cin>>a>>b;
}
void putdata()
{
 cout<<"\na="<<a
     <<"\nb="<<b;
}
```

```
void operator=(demo bb)
{
  a=bb.a;
  b=bb.a;
};
int main()
{
demo aa,bb;
bb.getdata();
aa=bb;
aa.putdata();
bb.putdata();
```

bb=aa

aa[a b

bb[a b

+=

a+=b

a=a+b

aa += bb

aa = aa+bb

```
class demo
 {int a;
  public:
   void getdata( )
   {
    cout <<"\n Enter no";
    cin >>a
   }
   void putdata( )
  {
   cout<<"\n a="<<a;
  }
void operator+=(demo bb)
   {a = a+ bb.a
   }
};
 int main ( )
   {
    demo aa bb ;
     aa.getdata( ); aa.putdata( );
     bb.getdata( ); bb.putdata( );
     aa+=bb;
    aa.putdata( );
    bb.putdata( );
    return 0;
   }
```

aa+=bb

aa=aa+bb

-=, *=, /=

// aa.a = aa.a + bb.a

aa.a = 4

bb.a = 2

calling += with ref. to aa and passing bb as argument

Virtual functions

Runtime or dynamic polymorphism.

Virtual function is a member function which is declared within a base class and is redefined by derived class.

When you refer to a derived class object using a pointer or a reference to the base class you can call a virtual funcn for that object and execute the derived class funcn.